

SDK / стек

Параметр	Значение
Название приложения	888Casino
Android Gradle Plugin	8.7.3
minSdk	26
targetSdk	36
Kotlin	да, 2.0.21
Web View	да
Custom Tabs	нет
Рекламные сети	нет
Аналитика	нет
Permissions	android.permission.INTERNET, android.permission.ACCESS_WIFI_STATE, android.permission.ACCESS_NETWORK_STATE, com.luckyreads.bookshelf.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, com.android.vending.CHECK_LICENSE
Libraries	Kotlin 2.0.21, kotlinx-coroutines 1.7.35, OkHttp 4.12.0, AndroidX (AppCompat, Activity, ConstraintLayout, Core/Core-KTX, Fragment, Lifecycle, Room, RecyclerView, ViewPager2, Emoji2, Startup, ProfileInstaller, CardView, CoordinatorLayout, DrawerLayout, Transition, VectorDrawable, SavedState, SQLite), Google Material Components, Pairip licensecheck (com.pairip.licensecheck)
Подозрительные домены	нет
SharedPreferences	файл prefs «cfg»: ключ «rurl» — сохранённый URL после редиректа с Cloudflare Worker; при повторном запуске сразу открывается через Intent VIEW
Есть ли клоака	да
Подозрительные слова	casino, WebView, User-Agent, rurl, redirect, license check, white page, Lucky Reads, workers.dev

Как работает клоака

В начале

Снаружи в Google Play приложение продаётся как «888Casino» (разработчик Zimvoler, пакет com.luckyreads.bookshelf). По названию это казино-бренд, но внутри APK лежит совсем другой продукт: локальная HTML-«читалка» классики под именем «Lucky Reads» (100 произведений, поиск, Lucky Pick) — её показывают в WebView (встроенном окне браузера внутри приложения).

Обман в том, что при первом запуске приложение не сразу открывает эту библиотеку. Сначала оно тихо спрашивает свой сервер на Cloudflare Workers (blue-limit-83a7.serg-savwork.workers.dev), можно ли этому пользователю показать «белую» читалку или увести его наружу. Сервер решает через HTTP-редирект (перенаправление на другой адрес): если ответа с другим URL нет — остаётся безопасная маска; если URL сменился — это «боевая» ссылка.

«Нужный» пользователь в итоге уходит из приложения во внешний браузер (через обычный Intent VIEW — системное «открыть ссылку») на динамический URL, который прислал Worker. Конкретный финальный сайт (казино/лендинг/оффер) в APK не зашит: он приезжает только с сервера. В domain_checks кастомных доменов для VirusTotal не отмечено (хост на workers.dev отфильтрован пайплайном как инфраструктура Cloudflare).

Кому что показывают

Белая страница / обычный режим. Её видит тот, кому Worker не делает редирект (типично: проверка магазина, бот, «нецелевой» IP/страна — решение на стороне сервера). Приложение показывает WebView с локальным файлом file:///android_asset/index.html — полноценный офлайн-каталог «Lucky Reads» (Browse All / Lucky Pick / Search). Никакой рекламы и никакого внешнего гемблинг-URL на экране нет. То же происходит при ошибке сети или если внешний браузер не удалось открыть.

Чёрный / боевой режим. Если после запроса к Worker итоговый URL отличается от исходного (сервер сделал редирект), приложение сохраняет этот URL в SharedPreferences под ключом «rurl» и сразу открывает его снаружи через Intent ACTION_VIEW. WebView при этом скрывают. При следующих запусках сохранённый «rurl» открывается снова, без повторного опроса (пока значение лежит в prefs).

Как приложение решает, кого куда пустить

- 1) Свой API на Cloudflare Worker. Что смотрит: GET `https://blue-limit-83a7.serg-savwork.workers.dev/?app=com.luckyreads.bookshelf` (OkHttp следует редиректам). Условие: сравнить исходный URL запроса и финальный URL ответа. Результат: совпали → белая HTML-читалка; отличаются → сохранить финальный URL и открыть снаружи.
- 2) User-Agent (строка браузера устройства). Что смотрит: заголовок User-Agent = `WebSettings.getDefaultUserAgent` (как у системного WebView). Условие: сервер может отличить «живой» Android WebView от бота/скрипта. Результат: на клиенте только передача; фильтр — на Worker (в коде не видно).
- 3) Модель устройства. Что смотрит: кастомный заголовок X-Device-Model = `Build.MODEL`. Условие/результат: дополнительный сигнал для серверной фильтрации (эмулятор vs реальное устройство) — логика только на бэкенде.
- 4) Язык / Accept-Language. Что смотрит: заголовок жёстко `en-US,en;q=0.9` (не берётся из настроек телефона). Условие: сервер видит «английский» клиент. Результат: вместе с IP запроса это типичный набор для geo/traffic-фильтров на Worker; в APK отдельных проверок страны нет.
- 5) IP / гео / VPN. Что смотрит: в клиентском коде нет списка стран и нет VPN-детекта. Условие и фильтр, если есть, — по IP на стороне Cloudflare Worker (в коде не видно). Результат: клиент просто сравнивает URL.
- 6) SharedPreferences «cfg» / ключ «rurl». Что смотрит: уже сохранённая боевая ссылка. Условие: строка не пустая. Результат: пропуск повторного запроса к Worker и сразу внешний Intent на этот URL (закрепление «чёрного» режима после первого успешного редиректа).
- 7) Pairip license check. Что смотрит: лицензия Google Play (CHECK_LICENSE) при старте Application. Условие: установка из Play. Результат: защита/привязка к магазину, а не выбор белый/чёрный контент.

Цепочка по шагам

- Шаг 1. Пользователь открывает приложение → стартует MainActivity с полноэкранным WebView → система готова либо показать локальную читалку, либо увести наружу.
- Шаг 2. Читаются SharedPreferences «cfg» → ищется ключ «rurl» → если URL уже сохранён, сразу вызывается внешнее открытие ссылки (без нового запроса к серверу).
- Шаг 3. Если «rurl» пуст → OkHttp собирает GET на Cloudflare Worker с `?app=com.luckyreads.bookshelf` → сервер получает package name и может выбрать сценарий под это приложение.
- Шаг 4. К запросу добавляют заголовки X-Device-Model, Accept-Language=en-US, User-Agent WebView → сервер видит «портрет» устройства → может отфильтровать ботов/модерацию/нужную гео.
- Шаг 5. Worker отвечает: либо 200 без смены URL (белый), либо HTTP-редирект на другой адрес (боевой) → OkHttp следует редиректам → клиент сравнивает исходный и финальный URL.
- Шаг 6. URL совпали → вызывается `v()`: WebView показывает `file:///android_asset/index.html` («Lucky Reads») → пользователь видит каталог классики.
- Шаг 7. URL различаются → новый адрес пишется в «rurl» → вызывается `w()`: WebView скрывают, стартует Intent VIEW → человек уходит во внешний браузер на динамический оффер/лендинг.
- Шаг 8. Сеть упала или Intent не открылся → запасной путь снова грузит локальный `index.html` → маска читалки не ломается даже при сбое.
- Шаг 9. При следующем запуске сохранённый «rurl» снова открывается снаружи → «боевой» пользователь закреплён без повторного «теста» Worker (пока не очистят данные приложения).

Технические уловки

XOR-шифрование строк. URL Worker, имя prefs «cfg», ключ «rurl», имена заголовков и путь к `index.html` лежат в байтовых массивах и расшифровываются методом `MainActivity.t()` ключом `[193,30,135,90]`. Зачем: чтобы в `strings.xml` и простом `grep` не светились адреса и смысл клоаки. В итоге расшифровывается полный endpoint `https://blue-limit-83a7.serg-savwork.workers.dev/?app=...` и `file:///android_asset/index.html`.

Детект редиректа вместо явного JSON-флага. Клиент не парсит «`show_offer=true`». Он смотрит, сменился ли URL после OkHttp follow-redirects. Зачем: логика решения полностью на сервере; в APK нет списка офферов.

Белая страница как локальный asset. `index.html` — автономное приложение-читалка (100 книг в JS). Зачем: модератор и бот видят живой продукт без сети и без подозрительного домена на экране.

Внешний Intent вместо Custom Tabs API. Боевой URL открывают через `android.intent.action.VIEW`, не через `androidx.browser.CustomTabs`. Зачем: увести трафик из процесса приложения в системный браузер; в

манифесте/коде Custom Tabs нет.

usesCleartextTraffic=true + JS/DOM Storage в WebView. Разрешён небезопасный HTTP и полноценный веб-рантайм для белой страницы. Pairip/LicenseActivity — обёртка Play license, не часть выбора оффера.

Роль подозрительных доменов

В таблице «Подозрительные домены» — «нет»: domain_checks.json / domain_checks.md зафиксировали, что кастомных хостов для отдельной VirusTotal-таблицы нет (all_custom_domains пуст). Отдельные таблицы «Проверка домена» поэтому не строятся.

При этом в коде всё равно фигурирует управляющий хост blue-limit-83a7.serg-savwork.workers.dev (Cloudflare Workers). Он появляется на шагах 3-5 цепочки: это «мозг» решения белый/чёрный (редирект или нет). Финальный лендинг с него не обязан совпадать с самим workers.dev — OkHttp может уйти на любой Location после редиректа. VT-вердикты по этому хосту пайплайн не приложил, выдумывать их нельзя.

Финал для пользователя

После всех проверок есть два исхода. «Нецелевой» / модераторский трафик остаётся внутри приложения на офлайн-читалке «Lucky Reads». «Боевой» пользователь уходит во внешний браузер на динамический URL из редиректа Worker; сам URL в APK не зашит и в domain_checks не зафиксирован. По бренду стор (888Casino) цель похожа на казино/гемблинг-оффер, но точный финальный сайт виден только в рантайме ответе сервера (или в уже сохранённом «rurl» на устройстве).